BANCO DE DADOS

com Microsoft SQL Server

AULA 10

Relacionamentos — INNER JOIN

Combinando dados de múltiplas tabelas

- O que é JOIN
- INNER JOIN
- Produto Cartesiano
- JOIN múltiplos
- JOIN + WHERE
- JOIN + GROUP BY
- Alias em JOIN
- Casos práticos

🎯 Objetivos da Aula 10

- 🔗 Entender o conceito de JOIN e por que ele existe
- 🔍 Combinar INNER JOIN com WHERE, ORDER BY e GROUP BY
- ⚠️ Conhecer o produto cartesiano e como evitá-lo
- ✏️ Usar aliases de tabela para tornar queries legíveis
- ⚡ Dominar o INNER JOIN: sintaxe e condição ON
- 📊 Criar relatórios que cruzam dados de múltiplas tabelas
- 📋 Unir 3, 4 e 5 tabelas em uma única query
- 💻 Explorar o SISTEMA_VENDAS com consultas relacionais reais

🔄 Revisão — Aula 09: Ordenação e Agrupamento

❓ Qual a diferença entre WHERE e HAVING?

✅ WHERE filtra LINHAS (antes do GROUP BY). HAVING filtra GRUPOS (depois do GROUP BY)

❓ Regra obrigatória do GROUP BY?

✅ Toda coluna não-agregada no SELECT deve aparecer no GROUP BY

❓ Por que WHERE não pode usar SUM ou COUNT?

✅ WHERE executa antes do GROUP BY — as funções de agregação ainda não foram calculadas

❓ O que faz ROLLUP?

✅ Gera subtotais automáticos em hierarquia + linha de total geral (NULL na coluna de grupo)

❓ Diferença entre RANK e DENSE_RANK?

✅ RANK tem lacunas nos empates (1,2,2,4). DENSE_RANK não tem lacunas (1,2,2,3)

Por que Precisamos de JOIN?

No modelo relacional, os dados são distribuídos em múltiplas tabelas para evitar redundância. JOIN é o mecanismo que recombina essas tabelas na hora da consulta, respeitando os relacionamentos definidos pelas FKs.

❌ Sem JOIN — dado incompleto

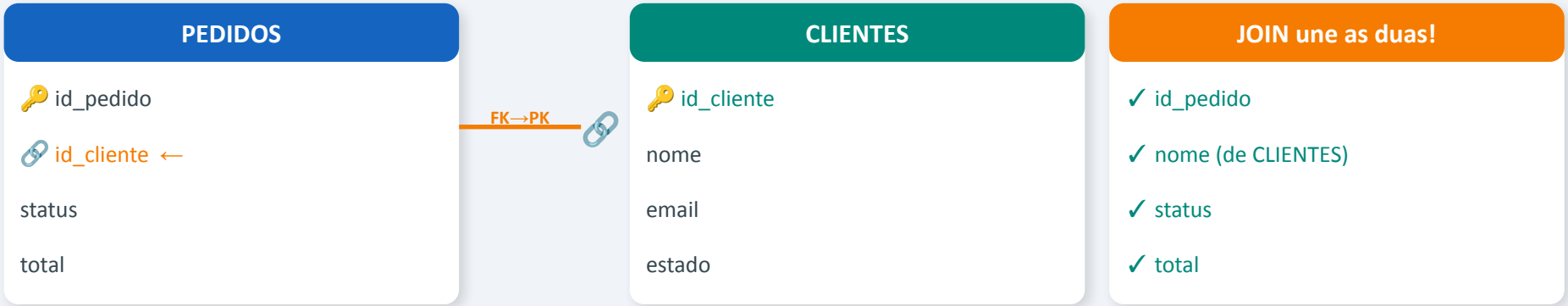
```
SELECT id_pedido, id_cliente, total
FROM   PEDIDOS;

-- Resultado: id_cliente=1, mas quem é?
-- Não sabemos o nome sem JOIN!
```

✅ Com JOIN — informação completa

```
SELECT p.id_pedido, c.nome, p.total
FROM   PEDIDOS   p
JOIN   CLIENTES  c ON p.id_cliente=c.id_cliente;

-- Agora sabemos: pedido 1 é da Ana Silva!
```



⚠️ Produto Cartesiano — O Erro Mais Perigoso

Produto cartesiano = combinação de TODAS as linhas de A com TODAS as linhas de B. Com 8 clientes e 10 produtos: 8 × 10 = 80 linhas inúteis! Em tabelas grandes, pode travar o servidor.

CLIENTES (3)
Ana
Bruno
Carla

✖

PRODUTOS (3)
iPhone
Camisa
Arroz

= 9 linhas!

Produto Cartesiano
Ana + iPhone
Ana + Camisa
Ana + Arroz
Bruno + iPhone
Bruno + Camisa
Bruno + Arroz
(9 no total)

❌ **Gera produto cartesiano:**

```
SELECT * FROM PEDIDOS, CLIENTES;  -- SEM condição de JOIN
```

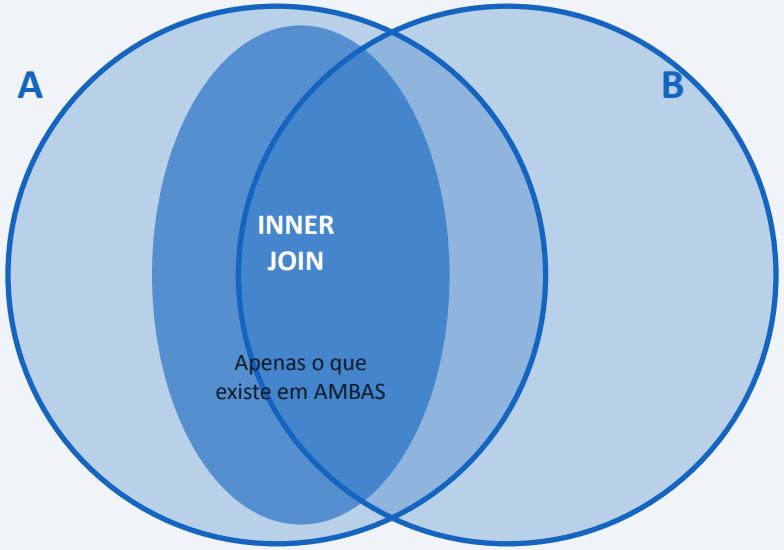
✅ **INNER JOIN (correto):**

```
SELECT * FROM PEDIDOS p JOIN CLIENTES c ON
p.id_cliente=c.id_cliente;
```

💡 **Sempre que você fizer JOIN entre N tabelas, verifique: tem N-1 condições ON? Se não, pode estar gerando produto cartesiano!**

INNER JOIN — Sintaxe e Funcionamento

INNER JOIN retorna apenas as linhas que têm correspondência nas DUAS tabelas. Se uma linha não tiver par na outra tabela, ela é EXCLUÍDA do resultado.



Formas equivalentes de escrever

✅ **Recomendado (ANSI SQL)**

```
SELECT p.id_pedido, c.nome
FROM   PEDIDOS p
INNER  JOIN CLIENTES c
       ON p.id_cliente = c.id_cliente;
```

✅ **Abreviado (JOIN = INNER JOIN)**

```
SELECT p.id_pedido, c.nome
FROM   PEDIDOS p
JOIN   CLIENTES c
       ON p.id_cliente = c.id_cliente;
```

⚠️ **Sintaxe antiga (evite)**

```
SELECT p.id_pedido, c.nome
FROM   PEDIDOS p, CLIENTES c
WHERE  p.id_cliente = c.id_cliente;
```

💻 INNER JOIN — Primeiros Exemplos no SISTEMA_VENDAS

```sql
USE SISTEMA_VENDAS;

-- Pedidos COM o nome do cliente
SELECT p.id_pedido,
       c.nome        AS cliente,
       p.status,
       p.total
FROM   PEDIDOS  p
INNER  JOIN CLIENTES c ON p.id_cliente = c.id_cliente
ORDER  BY p.data_pedido DESC;

-- Produtos COM o nome da categoria
SELECT pr.nome_produto,
       cat.nome_categoria,
       pr.preco,
       pr.estoque
FROM   PRODUTOS    pr
INNER  JOIN CATEGORIAS cat ON pr.id_categoria = cat.id_categoria
ORDER  BY cat.nome_categoria, pr.preco;
```

```sql
-- Itens de pedido COM nome do produto
SELECT ip.id_pedido,
       pr.nome_produto,
       ip.quantidade,
       ip.preco_unitario,
       ip.quantidade * ip.preco_unitario AS subtotal
FROM   ITENS_PEDIDO ip
INNER  JOIN PRODUTOS pr ON ip.id_produto = pr.id_produto
ORDER  BY ip.id_pedido, pr.nome_produto;
```

JOIN Múltiplos — 3, 4 e 5 Tabelas

```sql
-- 5 tabelas: relatório completo de vendas
SELECT p.id_pedido,
       FORMAT(p.data_pedido,'dd/MM/yyyy') AS data,
       c.nome        AS cliente,
       c.estado      AS uf,
       f.nome        AS vendedor,
       cat.nome_categoria AS categoria,
       pr.nome_produto,
       ip.quantidade,
       ip.quantidade * ip.preco_unitario AS subtotal,
       p.status
FROM   ITENS_PEDIDO ip
INNER  JOIN PEDIDOS    p   ON ip.id_pedido    = p.id_pedido
INNER  JOIN CLIENTES   c   ON p.id_cliente    = c.id_cliente
INNER  JOIN FUNCIONARIOS f  ON p.id_funcionario= f.id_funcionario
INNER  JOIN PRODUTOS   pr  ON ip.id_produto   = pr.id_produto
INNER  JOIN CATEGORIAS cat  ON pr.id_categoria = cat.id_categoria
ORDER  BY p.id_pedido, pr.nome_produto;
```

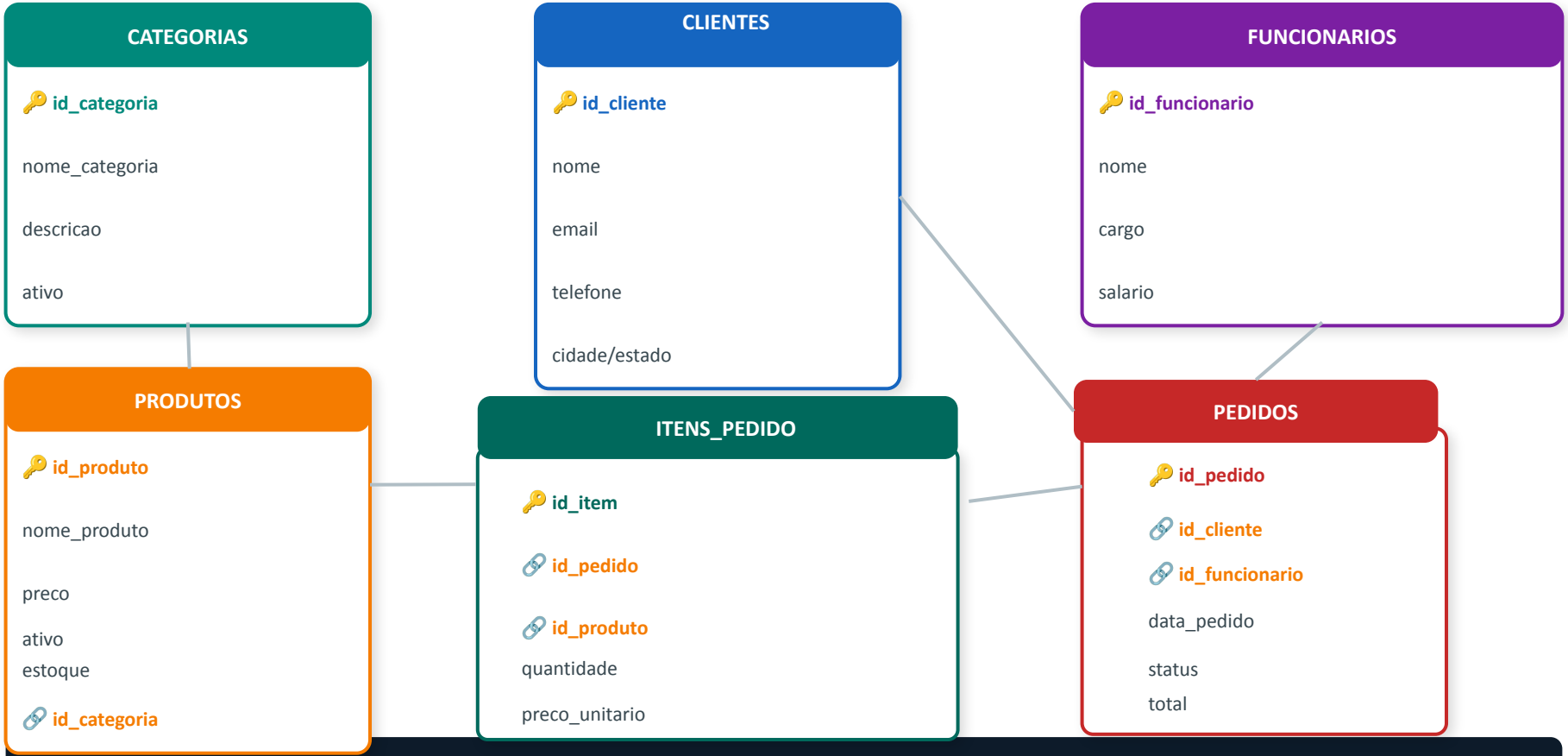
```sql
-- 3 tabelas: ITENS_PEDIDO + PEDIDOS + CLIENTES
SELECT p.id_pedido,
       c.nome        AS cliente,
       pr.nome_produto,
       ip.quantidade,
       ip.preco_unitario,
       ip.quantidade * ip.preco_unitario AS subtotal
FROM   ITENS_PEDIDO ip
INNER  JOIN PEDIDOS   p  ON ip.id_pedido  = p.id_pedido
INNER  JOIN CLIENTES  c  ON p.id_cliente  = c.id_cliente
INNER  JOIN PRODUTOS  pr ON ip.id_produto = pr.id_produto
ORDER  BY p.id_pedido, pr.nome_produto;
```

💻 JOIN + WHERE + GROUP BY — Relatórios Poderosos

```sql
-- Faturamento por categoria (JOIN + GROUP BY)
SELECT cat.nome_categoria,
       COUNT(DISTINCT p.id_pedido)          AS total_pedidos,
       SUM(ip.quantidade)                   AS itens_vendidos,
       SUM(ip.quantidade*ip.preco_unitario) AS faturamento
FROM   ITENS_PEDIDO ip
INNER  JOIN PRODUTOS   pr  ON ip.id_produto   = pr.id_produto
INNER  JOIN CATEGORIAS cat ON pr.id_categoria = cat.id_categoria
INNER  JOIN PEDIDOS    p   ON ip.id_pedido    = p.id_pedido
WHERE  p.status <> 'CANCELADO'
GROUP  BY cat.nome_categoria
ORDER  BY faturamento DESC;

-- Top 3 clientes por valor gasto (JOIN + GROUP BY + TOP)
SELECT TOP 3
       c.nome,
       c.estado,
       COUNT(p.id_pedido)  AS total_pedidos,
       SUM(p.total)        AS gasto_total
FROM   CLIENTES c
INNER  JOIN PEDIDOS p ON c.id_cliente = p.id_cliente
WHERE  p.status <> 'CANCELADO'
GROUP  BY c.nome, c.estado
ORDER  BY gasto_total DESC;
```

Mapa de JOINs — SISTEMA_VENDAS Completo



💻 JOIN + HAVING — Análises Avançadas

```sql
-- Categorias com faturamento acima de R$ 500
SELECT cat.nome_categoria,
       COUNT(DISTINCT p.id_pedido)           AS pedidos,
       SUM(ip.quantidade*ip.preco_unitario)  AS faturamento
FROM   ITENS_PEDIDO ip
INNER  JOIN PRODUTOS   pr  ON ip.id_produto   = pr.id_produto
INNER  JOIN CATEGORIAS cat ON pr.id_categoria = cat.id_categoria
INNER  JOIN PEDIDOS    p   ON ip.id_pedido    = p.id_pedido
WHERE  p.status <> 'CANCELADO'
GROUP  BY cat.nome_categoria
HAVING SUM(ip.quantidade*ip.preco_unitario) > 500
ORDER  BY faturamento DESC;

-- Vendedores com mais de 1 pedido entregue
SELECT f.nome          AS vendedor,
       f.cargo,
       COUNT(*)        AS pedidos_entregues,
       SUM(p.total)    AS faturamento
FROM   FUNCIONARIOS f
INNER  JOIN PEDIDOS p ON f.id_funcionario = p.id_funcionario
WHERE  p.status = 'ENTREGUE'
GROUP  BY f.nome, f.cargo
HAVING COUNT(*) > 0
ORDER  BY faturamento DESC;
```

💻 Casos Práticos — Relatórios Reais de Negócio

```sql
-- Detalhe completo de um pedido específico
SELECT p.id_pedido,
       FORMAT(p.data_pedido,'dd/MM/yyyy HH:mm') AS data,
       c.nome          AS cliente,
       f.nome          AS vendedor,
       p.status,
       pr.nome_produto AS produto,
       cat.nome_categoria,
       ip.quantidade,
       ip.preco_unitario,
       ip.quantidade * ip.preco_unitario AS subtotal
FROM   PEDIDOS      p
JOIN   CLIENTES     c   ON p.id_cliente    = c.id_cliente
JOIN   FUNCIONARIOS f   ON p.id_funcionario= f.id_funcionario
JOIN   ITENS_PEDIDO ip  ON ip.id_pedido    = p.id_pedido
JOIN   PRODUTOS     pr  ON ip.id_produto   = pr.id_produto
JOIN   CATEGORIAS   cat ON pr.id_categoria = cat.id_categoria
WHERE  p.id_pedido = 2;
```

```sql
-- Estoque crítico COM nome da categoria
SELECT cat.nome_categoria,
       pr.nome_produto,
       pr.estoque,
       CASE WHEN pr.estoque=0 THEN ' 🚨 SEM ESTOQUE'
            ELSE ' ⚠️ ESTOQUE BAIXO' END AS situacao
FROM   PRODUTOS    pr
JOIN   CATEGORIAS cat ON pr.id_categoria = cat.id_categoria
WHERE  pr.estoque < 15
ORDER  BY pr.estoque ASC;
```

✏️ Exercícios Propostos — Aula 10

01 — Básico

Use INNER JOIN para listar todos os produtos com o nome da sua categoria. Mostre: nome_produto, nome_categoria, preço e estoque. Ordene por categoria e depois por preço.

02 — Básico

Liste todos os pedidos com o nome do cliente, status e total. Filtre apenas pedidos com status 'ENTREGUE' ou 'ENVIADO'. Ordene pelo total decrescente.

03 — Médio

Crie um relatório de itens de pedido com: id_pedido, nome do cliente, nome do produto, quantidade, preço_unitario e subtotal (calculado). Use 3 tabelas: ITENS_PEDIDO + PEDIDOS + CLIENTES + PRODUTOS.

04 — Médio

Faturamento por vendedor: JOIN entre PEDIDOS e FUNCIONARIOS. Agrupe por nome do funcionário, some o total (excluindo cancelados), mostre COUNT de pedidos e ticket médio. Ordene por faturamento desc.

05 — Avançado

Relatório completo: para cada categoria, mostre o faturamento total (SUM de itens), quantidade de pedidos distintos e o produto mais vendido (subquery ou HAVING). Use 5 tabelas. Filtre categorias com faturamento > 0.

✅ Respostas Comentadas — Exercícios 03 e 04

```sql
-- EX 03: Itens de pedido com 4 tabelas
SELECT p.id_pedido,
       c.nome          AS cliente,
       pr.nome_produto AS produto,
       ip.quantidade,
       ip.preco_unitario,
       ip.quantidade * ip.preco_unitario AS subtotal
FROM   ITENS_PEDIDO ip
INNER  JOIN PEDIDOS   p  ON ip.id_pedido  = p.id_pedido
INNER  JOIN CLIENTES  c  ON p.id_cliente  = c.id_cliente
INNER  JOIN PRODUTOS  pr ON ip.id_produto = pr.id_produto
ORDER  BY p.id_pedido, pr.nome_produto;

-- EX 04: Faturamento por vendedor com JOIN + GROUP BY
SELECT f.nome                          AS vendedor,
       f.cargo,
       COUNT(p.id_pedido)              AS total_pedidos,
       SUM(CASE WHEN p.status <> 'CANCELADO'
                THEN p.total ELSE 0
           END)                        AS faturamento,
       ROUND(AVG(CASE WHEN p.status<>'CANCELADO'
                      THEN p.total END),2) AS ticket_medio
FROM   FUNCIONARIOS f
INNER  JOIN PEDIDOS p ON f.id_funcionario = p.id_funcionario
GROUP  BY f.nome, f.cargo
ORDER  BY faturamento DESC;
```

🏆 Desafio da Aula 10

📋 Contexto: O diretor quer o relatório de desempenho mais completo que a empresa já teve — cruzando dados de clientes, vendedores, produtos, categorias e pedidos em um único painel.

🎯 Relatórios solicitados:

R1 Nota fiscal digital: para o pedido 2, liste todos os itens com nome do produto, categoria, quantidade, preço e subtotal. Inclua o total do pedido no final com UNION.

R2 Ranking de clientes: nome, estado, total de pedidos (não cancelados), gasto total e ticket médio. TOP 5 por gasto total. JOIN CLIENTES+PEDIDOS+GROUP BY.

R3 Mix de produtos vendidos: para cada produto vendido, mostre nome, categoria, quantidade total vendida e faturamento. Apenas produtos com pelo menos 1 venda. Use 4 tabelas.

R4 Painel do vendedor: para cada vendedor, mostre pedidos entregues/enviados/pendentes/cancelados (COUNT CASE) e faturamento real. Use JOIN FUNCIONARIOS+PEDIDOS.

R5 BÔNUS: Relatório cruzado — por estado do cliente, por categoria: faturamento total. Use todas as 6 tabelas. GROUP BY c.estado, cat.nome_categoria. ORDER BY estado, faturamento DESC.

📚 Resumo — Aula 10: INNER JOIN

🔗 Por que JOIN?

Dados distribuídos em tabelas relacionadas. JOIN recombina na hora da consulta

⚠️ Produto cartesiano

Sem condição ON: NxM linhas inúteis. Com N tabelas: precisa de N-1 condições ON

⚡ INNER JOIN

Retorna só linhas com correspondência nas DUAS tabelas. JOIN sem INNER = INNER JOIN

📋 Múltiplos JOINs

Encadeie quantos precisar. Tabela central → JOIN em todas as direções

✏️ Alias de tabela

p, c, pr, f, cat — obrigatório quando colunas existem em múltiplas tabelas

📊 JOIN + GROUP BY

COUNT(DISTINCT id) para não contar o mesmo registro N vezes em JOINs 1:N

🔍 WHERE antes do JOIN

Filtre com WHERE para reduzir dados antes do agrupamento — mais eficiente

💡 Próximos JOINs

INNER = correspondência em ambas. Próxima aula: LEFT, RIGHT, FULL, SELF JOIN

Próxima Aula 11: JOINs Avançados

Aula 11 — O que vem por aí:

LEFT JOIN	Inclui TODOS os registros da esquerda, mesmo sem correspondência à direita
RIGHT JOIN	Inclui TODOS os registros da direita, mesmo sem correspondência à esquerda
FULL JOIN	Inclui TODOS de ambos os lados — com e sem correspondência
SELF JOIN	Uma tabela se juntando a ela mesma — hierarquias e comparações
INNER vs LEFT	Quando usar cada um — impacto nos resultados
Casos práticos	Clientes sem pedido, funcionários sem venda, produtos sem categoria

📝 **Para a próxima aula: conclua o relatório de desempenho e pense: quais registros o INNER JOIN pode estar 'escondendo' nas suas consultas?**